Aizza Imran Bachlani
26k-2551
BDS-1B

Task 1:

```c
#include <stdio.h>
int main() {
    int a[3][3], transpose[3][3], cofactor[3][3], adjoint[3][3], minor[2][2];
    int i, j, r, c, x, y, det;
    printf("Enter the elements of 3x3 matrix:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            scanf("%d", &a[i][j]);
        }
    }
    printf("\nOriginal Matrix:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d\t", a[i][j]);
        }
        printf("\n");
    }
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            transpose[i][j] = a[j][i];
        }
    }
    printf("\nTranspose Matrix:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d\t", transpose[i][j]);
        }
        printf("\n");
    }
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            x = 0;
            for (r = 0; r < 3; r++) {
```

```
        for (j = 0; j < 3; j++) {
            x = 0;
            for (r = 0; r < 3; r++) {
                for (c = 0; c < 3; c++) {
                    if (r != i && c != j) {
                        minor[x / 2][x % 2] = a[r][c];
                        x++;
                    }
                }
            }
            cofactor[i][j] = (minor[0][0] * minor[1][1] - minor[0][1] * minor[1][0]);

            if ((i + j) % 2 != 0) {
                cofactor[i][j] = -cofactor[i][j];
            }
        }
    }
    printf("\nCofactor Matrix:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d\t", cofactor[i][j]);
        }
        printf("\n");
    }
    det = 0;
    for (j = 0; j < 3; j++) {
        det = det + a[0][j] * cofactor[0][j];
    }
    printf("\nDeterminant = %d\n", det);
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            adjoint[i][j] = cofactor[j][i];
        }
    }
    printf("\nAdjoint Matrix:\n");
```

```c
    printf("\nDeterminant = %d\n", det);
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            adjoint[i][j] = cofactor[j][i];
        }
    }
    printf("\nAdjoint Matrix:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d\t", adjoint[i][j]);
        }
        printf("\n");
    }
    if (det == 0) {
        printf("\nInverse does not exist because determinant is zero.\n");
    } else {
        printf("\nInverse Matrix:\n");
        for (i = 0; i < 3; i++) {
            for (j = 0; j < 3; j++) {
                printf("%.3f\t", (float)adjoint[i][j] / det);
            }
            printf("\n");
        }
    }
    return 0;
```

```
Enter the elements of 3x3 matrix:
4
5
5
5
6
5
1
2
5

Original Matrix:
4       5       5
5       6       5
1       2       5

Transpose Matrix:
4       5       1
5       6       2
5       5       5

Cofactor Matrix:
20      -20     4
-15     15      -3
-5      5       -1

Determinant = 0

Adjoint Matrix:
20      -15     -5
-20     15      5
4       -3      -1

Inverse does not exist because determinant is zero.

--------------------------------
```

Task 2:

```c
#include <stdio.h>
#include <math.h>
int main()
{
    int a[5][5], b[5][5], power[5][5], temp[5][5];
    int r, c, i, j, k, choice;
    int zero = 1, identity = 1, diagonal = 1, scalar = 1, upper = 1, lower = 1, symmetric = 1, skew = 1, equal = 1;
    int idempotent = 1, nilpotent = 0;
    int rowMatrix, columnMatrix, det = 0;
    printf("Enter number of rows (1-5): ");
    scanf("%d", &r);
    printf("Enter number of columns (1-5): ");
    scanf("%d", &c);
    if(r<1 || r>5 || c<1 || c>5){
        printf("Invalid matrix size!");
        return 0;
    }
    printf("Enter matrix elements:\n");
    for(i=0; i<r; i++){
        for (j=0; j<c; j++){
            scanf("%d", &a[i][j]);
        }
    }
    printf("\nMatrix:\n");
    for(i=0; i<r; i++){
        for (j = 0; j < c; j++){
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }
    for(i = 0; i < r; i++){
        for (j = 0; j < c; j++){
            if (a[i][j] != 0)
                zero = 0;
        }
    }
    if (r == c)
```

```
34                  zero = 0;
35              }
36          }
37          if (r == c)
38              printf("\nSquare Matrix");
39          else
40              printf("\nRectangular Matrix");
41
42          if (zero)
43              printf("\nZero Matrix");
44          else
45              printf("\nNot a Zero Matrix");
46          if (zero)
47              printf("\nNull Matrix");
48          else
49              printf("\nNot a Null Matrix");
50          rowMatrix = (r == 1);
51          columnMatrix = (c == 1);
52          if (rowMatrix)
53              printf("\nRow Matrix");
54          if (columnMatrix)
55              printf("\nColumn Matrix");
56          if (r == c){
57              for (i = 0; i < r; i++){
58                  for (j = 0; j < c; j++){
59                      if (i != j && a[i][j] != 0)
60                          diagonal = 0;
61                      if (i == j){
62                          if (a[i][j] != 1)
63                              identity = 0;
64                          if (i > 0 && a[i][i] != a[0][0])
65                              scalar = 0;
66                      }
67                      if (i < j && a[i][j] != 0)
68                          lower = 0;
69                      if (i > j && a[i][j] != 0)
70                          upper = 0;
```

```
70                      upper = 0;
71                  if (a[i][j] != a[j][i])
72                      symmetric = 0;
73                  if (a[i][j] != -a[j][i])
74                      skew = 0;
75              }
76          }
77          if (identity)
78              printf("\nIdentity Matrix");
79          else
80              printf("\nNot an Identity Matrix");
81          if (diagonal)
82              printf("\nDiagonal Matrix");
83          else
84              printf("\nNot a Diagonal Matrix");
85          if (diagonal && scalar)
86              printf("\nScalar Matrix");
87          else
88              printf("\nNot a Scalar Matrix");
89          if (upper)
90              printf("\nUpper Triangular Matrix");
91          else
92              printf("\nNot an Upper Triangular Matrix");
93          if (lower)
94              printf("\nLower Triangular Matrix");
95          else
96              printf("\nNot a Lower Triangular Matrix");
97          if (symmetric)
98              printf("\nSymmetric Matrix");
99          else
100             printf("\nNot a Symmetric Matrix");
101         if (skew)
102             printf("\nSkew-Symmetric Matrix");
103         else
104             printf("\nNot a Skew-Symmetric Matrix");
105         {
106             int m[5][5], sub[5][5];
```

```
106                 int m[5][5], sub[5][5];
107                 int n, x, y, rr, cc, sign;
108                 int d[5];
109                 for (i = 0; i < r; i++)
110                     for (j = 0; j < c; j++)
111                         m[i][j] = a[i][j];
112                 n = r;
113                 if (n == 1)
114                     det = m[0][0];
115                 else if (n == 2)
116                     det = m[0][0] * m[1][1] - m[0][1] * m[1][0];
117                 else{
118                     int stack[100][5][5];
119                     int sizes[100], rows[100], cols[100];
120                     int signs[100], vals[100];
121                     int top = 0, result = 0;
122                     for (i = 0; i < n; i++)
123                         for (j = 0; j < n; j++)
124                             stack[0][i][j] = m[i][j];
125                     sizes[0] = n;
126                     rows[0] = 0;
127                     cols[0] = 0;
128                     signs[0] = 1;
129                     vals[0] = 0;
130                     while (top >= 0){
131                         int size = sizes[top];
132                         if (size == 1){
133                             result += signs[top] * stack[top][0][0];
134                             top--;
135                             continue;
136                         }
137                         if (cols[top] >= size){
138                             top--;
139                             continue;
140                         }
141                         x = cols[top]++;
142                         sign = (x % 2 == 0) ? 1 : -1;
```

```
142                     sign = (x % 2 == 0) ? 1 : -1;
143                     for (rr = 1; rr < size; rr++){
144                         cc = 0;
145                         for (y = 0; y < size; y++){
146                             if (y != x){
147                                 sub[rr - 1][cc] =
148                                     stack[top][rr][y];
149                                 cc++;
150                             }
151                         }
152                     }
153                     if (size == 3){
154                         int v = stack[top][0][x] * (sub[0][0] * sub[1][1] - sub[0][1] * sub[1][0]);
155                         result += signs[top] * sign * v;
156                     }
157                     else{
158                         break;
159                     }
160                 }
161                 if (n == 3)
162                     det = a[0][0] * (a[1][1] * a[2][2] - a[1][2] * a[2][1]) - a[0][1] * (a[1][0] * a[2][2]
163                             - a[1][2] * a[2][0]) + a[0][2] * (a[1][0] * a[2][1] - a[1][1] * a[2][0]);
164                 if (n >= 4){
165                     double f[5][5], factor, dd = 1;
166                     int signChange = 1, pivot;
167                     for (i = 0; i < n; i++)
168                         for (j = 0; j < n; j++)
169                             f[i][j] = a[i][j];
170                     for (i = 0; i < n; i++){
171                         pivot = i;
172                         for (j = i + 1; j < n; j++)
173                             if (fabs(f[j][i]) >
174                                 fabs(f[pivot][i]))
175                                 pivot = j;
176                         if (fabs(f[pivot][i]) < 0.000001){
177                             dd = 0;
178                             break;
```

```
178                         break;
179                     }
180                     if (pivot != i){
181                         for (k = 0; k < n; k++){
182                             double t = f[i][k];
183                             f[i][k] = f[pivot][k];
184                             f[pivot][k] = t;
185                         }
186                         signChange = -signChange;
187                     }
188                     dd *= f[i][i];
189                     for (j = i + 1; j < n; j++){
190                         factor = f[j][i] / f[i][i];
191                         for (k = i + 1; k < n; k++)
192                             f[j][k] -= factor * f[i][k];
193                     }
194                 }
195                 det = (int)(dd * signChange);
196             }
197         }
198     }
199     printf("\nDeterminant = %d", det);
200     if (det == 0)
201         printf("\nSingular Matrix");
202     else
203         printf("\nNon-Singular Matrix");
204     for (i = 0; i < r; i++){
205         for (j = 0; j < c; j++){
206             temp[i][j] = 0;
207             for (k = 0; k < r; k++)
208                 temp[i][j] += a[i][k] * a[k][j];
209             if (temp[i][j] != a[i][j])
210                 idempotent = 0;
211         }
212     }
213     if (idempotent)
214         printf("\nIdempotent Matrix");
```

```
216             printf("\nNot an Idempotent Matrix");
217         for (i = 0; i < r; i++)
218             for (j = 0; j < c; j++)
219                 power[i][j] = a[i][j];
220         for (int p = 2; p <= r; p++){
221             int allZero = 1;
222             for (i = 0; i < r; i++){
223                 for (j = 0; j < r; j++){
224                     temp[i][j] = 0;
225                     for (k = 0; k < r; k++)
226                         temp[i][j] += power[i][k] * a[k][j];
227                 }
228             }
229             for (i = 0; i < r; i++)
230                 for (j = 0; j < r; j++){
231                     power[i][j] = temp[i][j];
232                     if (power[i][j] != 0)
233                         allZero = 0;
234                 }
235             if (allZero)
236                 nilpotent = 1;
237         }
238         if (nilpotent)
239             printf("\nNilpotent Matrix");
240         else
241             printf("\nNot a Nilpotent Matrix");
242     }
243     else{
244         printf("\nOther square-matrix properties "
245                "cannot be checked.");
246     }
247     printf("\n\nDo you want to compare another matrix?");
248     printf("\nEnter 1 for Yes, 0 for No: ");
249     scanf("%d", &choice);
250     if (choice == 1){
251         printf("Enter second matrix (%d x %d):\n", r, c);
252         for (i = 0; i < r; i++)
253             for (j = 0; j < c; j++)
254                 scanf("%d", &b[i][j]);
255         equal = 1;
256         for (i = 0; i < r; i++)
257             for (j = 0; j < c; j++)
258                 if (a[i][j] != b[i][j])
259                     equal = 0;
260         if (equal)
261             printf("Equal Matrices");
262         else
263             printf("Not Equal Matrices");
264     }
265     return 0;
266 }
```

```
Enter number of rows (1-5): 4
Enter number of columns (1-5): 4
Enter matrix elements:
4
5
4
7
5
5
8
9
1
4
5
5
4
5
45
5

Matrix:
4 5 4 7
5 5 8 9
1 4 5 5
4 5 45 5

Square Matrix
Not a Zero Matrix
Not a Null Matrix
Not an Identity Matrix
Not a Diagonal Matrix
Not a Scalar Matrix
Not an Upper Triangular Matrix
Not a Lower Triangular Matrix
Not a Symmetric Matrix
Not a Skew-Symmetric Matrix
Determinant = 885
Non-Singular Matrix
Not an Idempotent Matrix
Not a Nilpotent Matrix

Do you want to compare another matrix?
Enter 1 for Yes, 0 for No:
```

Task 3:

```c
#include <stdio.h>
int main(){
    int a[3][3], b[3][3], result[3][3];
    int r1, c1, r2, c2, i, j, k;
    printf("Enter rows and columns of first matrix: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns of second matrix: ");
    scanf("%d %d", &r2, &c2);
    if (r1 < 1 || r1 > 3 || c1 < 1 || c1 > 3 || r2 < 1 || r2 > 3 || c2 < 1 || c2 > 3){
        printf("Invalid dimensions!");
        return 0;
    }
    if (c1 != r2){
        printf("Matrix multiplication is not possible.");
        return 0;
    }
    printf("Enter first matrix:\n");
    for (i = 0; i < r1; i++)
        for (j = 0; j < c1; j++)
            scanf("%d", &a[i][j]);
    printf("Enter second matrix:\n");
    for (i = 0; i < r2; i++)
        for (j = 0; j < c2; j++)
            scanf("%d", &b[i][j]);
    for (i = 0; i < r1; i++){
        for (j = 0; j < c2; j++){
            result[i][j] = 0;
            for (k = 0; k < c1; k++)
                result[i][j] += a[i][k] * b[k][j];
        }
    }
    printf("\nFirst Matrix:\n");
    for (i = 0; i < r1; i++){
        for (j = 0; j < c1; j++)
            printf("%d ", a[i][j]);
```

```
27              result[i][j] = 0;
28              for (k = 0; k < c1; k++)
29                  result[i][j] += a[i][k] * b[k][j];
30          }
31      }
32      printf("\nFirst Matrix:\n");
33      for (i = 0; i < r1; i++){
34          for (j = 0; j < c1; j++)
35              printf("%d ", a[i][j]);
36          printf("\n");
37      }
38      printf("\nSecond Matrix:\n");
39      for (i = 0; i < r2; i++){
40          for (j = 0; j < c2; j++)
41              printf("%d ", b[i][j]);
42          printf("\n");
43      }
44      printf("\nResultant Matrix:\n");
45      for (i = 0; i < r1; i++){
46          for (j = 0; j < c2; j++)
47              printf("%d ", result[i][j]);
48          printf("\n");
49      }
50      return 0;
51  }
```

```
Enter rows and columns of first matrix: 3
3
Enter rows and columns of second matrix: 3
3
Enter first matrix:
5
4
9
8
7
6
3
2
1
Enter second matrix:
4
5
6
1
2
3
1
1
3

First Matrix:
5 4 9
8 7 6
3 2 1

Second Matrix:
4 5 6
1 2 3
1 1 3

Resultant Matrix:
33 42 69
45 60 87
15 20 27

--------------------------------
Process exited after 29.98 seconds with return value 0
Press any key to continue . . .
```

Task 04:

```c
#include <stdio.h>
int main(){
    int a[3][3][3];
    int i, j, k, min, max;
    int layerTotal, total = 0;
    float average;
    printf("Enter 27 elements of the 3D array:\n");
    for (i = 0; i < 3; i++){
        for (j = 0; j < 3; j++){
            for (k = 0; k < 3; k++){
                scanf("%d", &a[i][j][k]);
            }
        }
    }
    min = a[0][0][0];
    max = a[0][0][0];
    for (i = 0; i < 3; i++){
        layerTotal = 0;
        printf("\nLayer %d:\n", i + 1);
        for (j = 0; j < 3; j++){
            for (k = 0; k < 3; k++){
                printf("%d ", a[i][j][k]);
                layerTotal += a[i][j][k];
                total += a[i][j][k];
                if (a[i][j][k] < min)
                    min = a[i][j][k];
                if (a[i][j][k] > max)
                    max = a[i][j][k];
            }
            printf("\n");
        }
        printf("Layer Total = %d\n", layerTotal);
        printf("Layer Average = %.2f\n",
                layerTotal / 9.0);
    }
    average = total / 27.0;
    printf("\nOverall Total = %d\n", total);
    printf("Overall Average = %.2f\n", average);
    printf("Minimum Element = %d\n", min);
    printf("Maximum Element = %d\n", max);
    return 0;
}
```

```
7
5
5
7
5
56
8
4
5
4
5
5

Layer 1:
4 5 8
45 5 5
1 2 3
Layer Total = 78
Layer Average = 8.67

Layer 2:
3 6 4
8 8 4
7 5 5
Layer Total = 50
Layer Average = 5.56

Layer 3:
7 5 56
8 4 5
4 5 5
Layer Total = 99
Layer Average = 11.00

Overall Total = 227
Overall Average = 8.41
Minimum Element = 1
Maximum Element = 56
```

Task 5:

```c
#include <stdio.h>
int main(){
    int a[3][3][3];
    int i, j, k, same;
    printf("Enter 27 elements of the 3D array:\n");
    for (i = 0; i < 3; i++){
        printf("\nEnter elements of Layer %d:\n", i + 1);
        for (j = 0; j < 3; j++){
            for (k = 0; k < 3; k++){
                scanf("%d", &a[i][j][k]);
            }
        }
    }
    printf("\nLayer Comparisons:\n");
    for (i = 0; i < 3; i++){
        for (j = i + 1; j < 3; j++){
            same = 1;
            for (int r = 0; r < 3; r++){
                for (int c = 0; c < 3; c++){
                    if (a[i][r][c] != a[j][r][c])
                        same = 0;
                }
            }
            if (same){
                printf("Layer %d and Layer %d are IDENTICAL.\n",
                       i + 1, j + 1);
            }
            else{
                printf("Layer %d and Layer %d are DISTINCT.\n",
                       i + 1, j + 1);
            }
        }
    }
    return 0;
}
```

```
Enter 27 elements of the 3D array:

Enter elements of Layer 1:
1
2
3
1
2
3
1
2
3

Enter elements of Layer 2:
1
2
3
1
2
3
1
2
3

Enter elements of Layer 3:
4
5
6
1
2
3
4
5
6

Layer Comparisons:
Layer 1 and Layer 2 are IDENTICAL.
Layer 1 and Layer 3 are DISTINCT.
Layer 2 and Layer 3 are DISTINCT.

--------------------------------
Process exited after 31.05 seconds with return value 0
Press any key to continue . . .
```